

Laporan DAST ADVISE

PENGEMBANGAN FITUR APLIKASI



Disusun Oleh:

Angela Vania Sugiyono 5025241226

Departemen Teknik Informatika, FTEIC
Institut Teknologi Sepuluh Nopember

2026

Daftar Isi

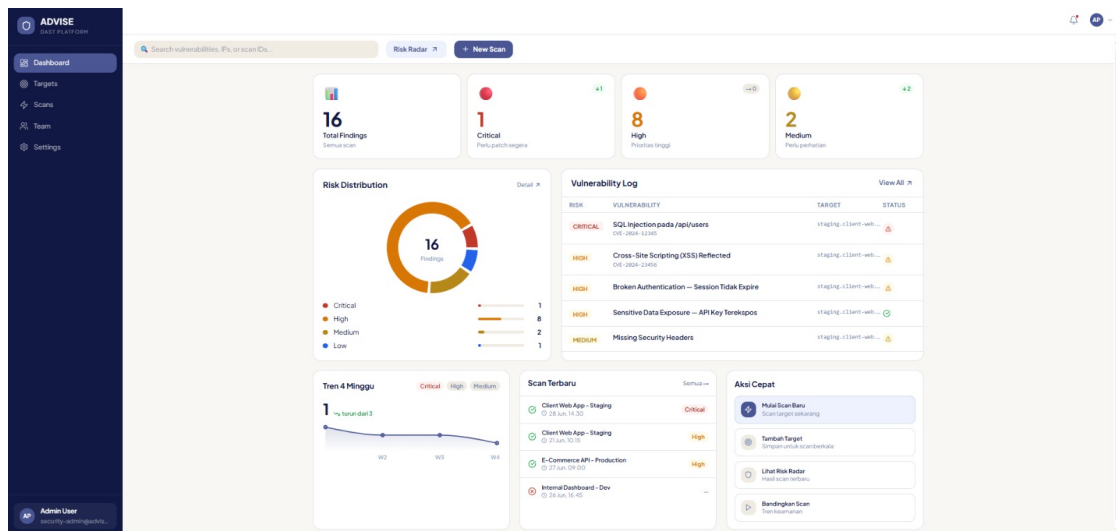
1	Penjelasan Fitur	2
1.1	Dashboard (Dasbor Utama)	2
1.2	Target (Manajemen Target)	2
1.3	Scans (Pemindaian)	3
1.4	Kerentanan (Vulnerabilities)	3
2	Analisis Urgensi	4
3	Metodologi AI	4
3.1	Ideasi dan Konseptualisasi	4
3.2	Perancangan UI/UX	5
3.3	Pembuatan Kode (<i>Code Generation</i>)	6

1 Penjelasan Fitur

Aplikasi DAST "Advise" (Dynamic Application Security Testing) dirancang untuk membantu pengembang dan profesional keamanan dalam memindai, mendeteksi, dan mengelola kerentanan pada aplikasi web secara otomatis. Berikut adalah detail fungsionalitas, cara kerja, dan alur purwarupa fitur yang telah dikembangkan berdasarkan antarmuka yang dipilih untuk memenuhi kebutuhan operasional:

1.1 Dashboard (Dasbor Utama)

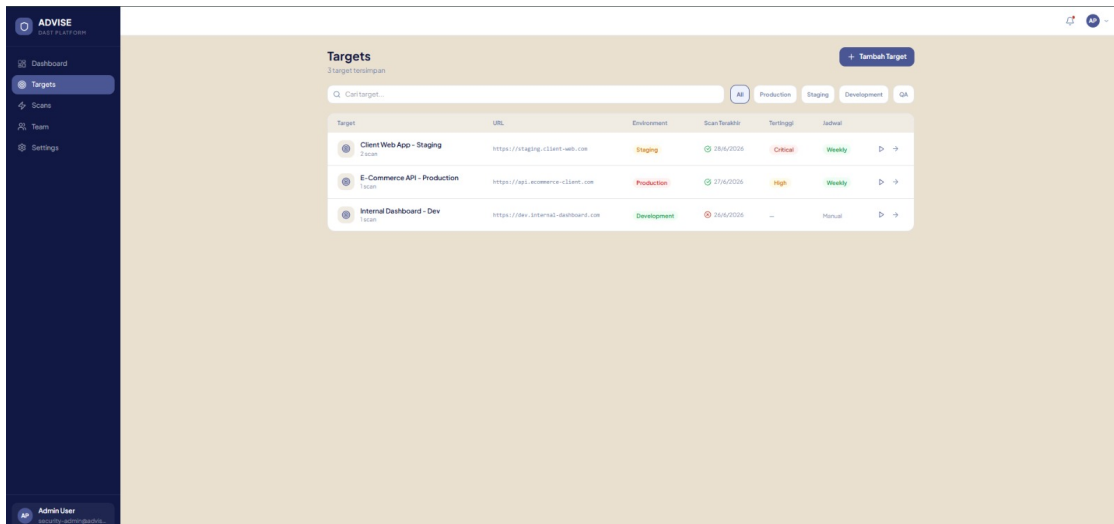
Fitur Dashboard berfungsi sebagai pusat kendali yang memberikan ringkasan (*overview*) dari postur keamanan aplikasi yang dipindai. Pada halaman ini, pengguna dapat melihat metrik utama seperti total pemindaian yang dilakukan, grafik tren penemuan kerentanan berdasarkan tingkat keparahan (*severity*), dan daftar aktivitas terbaru. Cara kerjanya adalah dengan menarik data dari pemindaian terakhir dan menyajikannya dalam bentuk visualisasi yang mudah dipahami. Desain ini dipilih karena pengguna membutuhkan informasi komprehensif secara sekilas agar dapat langsung mengetahui status keamanan sistem mereka.



Gambar 1: Antarmuka Dashboard DAST "Advise"

1.2 Target (Manajemen Target)

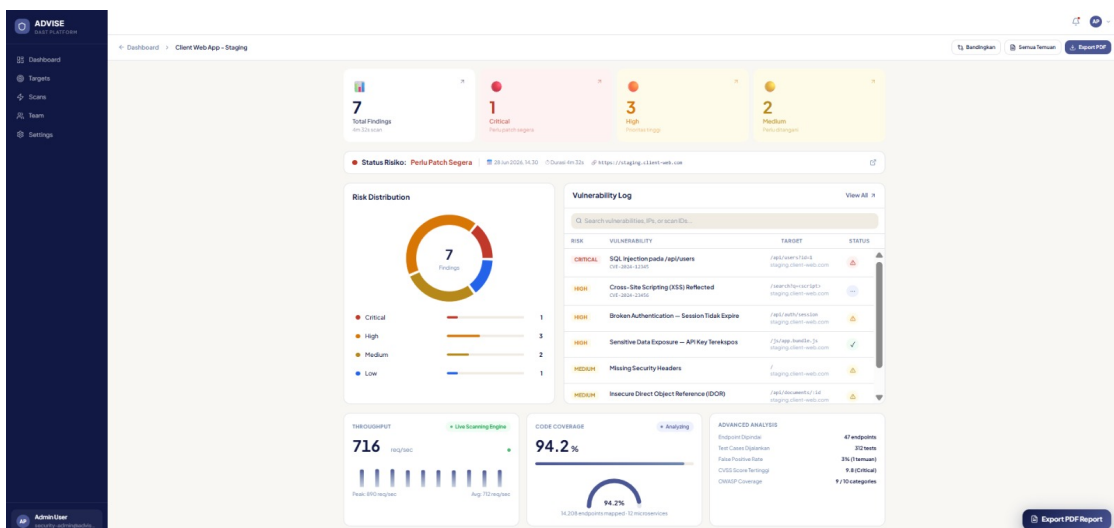
Fitur Target memungkinkan pengguna untuk mendaftarkan dan mengelola URL aplikasi web atau API yang akan menjadi target pemindaian. Pengguna dapat menambahkan target baru, mengonfigurasi autentikasi jika diperlukan, serta memvalidasi kepemilikan domain. Alur kerjanya: Pengguna memasukkan URL target, sistem memvalidasi konektivitas, dan target tersebut kemudian tersedia untuk dipilih saat pengguna hendak memulai proses pemindaian baru. Desain ini dipilih untuk memudahkan pengelolaan banyak *endpoint* secara terpusat.



Gambar 2: Antarmuka Manajemen Target

1.3 Scans (Pemindaian)

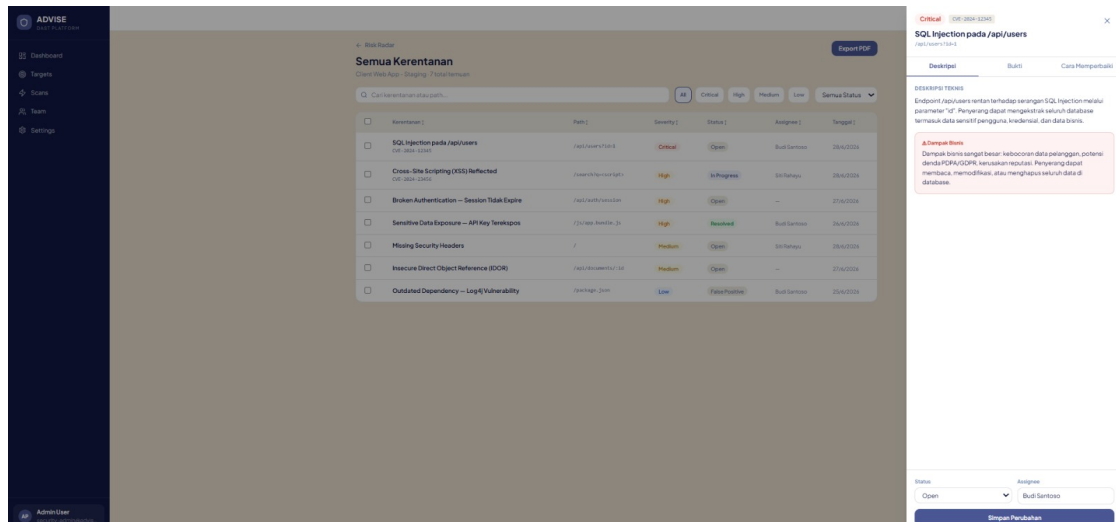
Fitur Scans adalah inti dari aplikasi ini, di mana pengguna dapat menginisiasi pemindaian keamanan baru, menjadwalkan pemindaian rutin, dan melihat riwayat pemindaian yang sedang atau telah berjalan. Cara kerjanya: Pengguna memilih target yang telah didaftarkan, menentukan profil pemindaian, dan memulai proses. Sistem akan mengeksekusi serangkaian *payload* pengujian ke target untuk mencari celah keamanan. Pendekatan antarmuka ini dipilih untuk memisahkan konfigurasi pemindaian dengan eksekusi sehingga pengguna tidak bingung.



Gambar 3: Antarmuka Riwayat dan Konfigurasi Scans

1.4 Kerentanan (Vulnerabilities)

Fitur Kerentanan menyajikan daftar lengkap kelemahan keamanan yang ditemukan dari hasil pemindaian. Setiap kerentanan diklasifikasikan berdasarkan tingkat keparahannya (Kritis, Tinggi, Sedang, Rendah). Cara kerjanya: Saat pengguna mengklik salah satu kerentanan, sistem akan menampilkan detail teknis mengenai celah tersebut, letak URL yang rentan, serta saran remediasi (*advise*) untuk memperbaikinya. Desain ini dirancang sedemikian rupa agar perbaikan dapat langsung dieksekusi oleh developer tanpa perlu mencari referensi luar.



Gambar 4: Antarmuka Daftar Kerentanan

2 Analisis Urgensi

Perancangan fitur-fitur pada aplikasi DAST "Advise" didasari oleh kebutuhan mendesak akan otomatisasi dan visibilitas dalam pengujian keamanan aplikasi. Berikut adalah alasan dan bagaimana fitur ini memberikan nilai tambah yang strategis bagi pengguna alat DAST:

- **Visibilitas Menyeluruh:** Melalui Dashboard dan daftar Kerentanan yang terstruktur, tim keamanan dan *developer* memiliki visibilitas langsung terhadap status keamanan seluruh aset digital mereka. Hal ini mengurangi waktu respons (*Mean Time to Respond*) terhadap celah keamanan kritis.
- **Otomatisasi Pengujian:** Fitur Scans memungkinkan pemindaian dilakukan secara otomatis dan berkala, mengintegrasikan praktik *Continuous Security* tanpa harus bergantung pada pengujian penetrasi manual yang memakan waktu dan biaya besar.
- **Rekomendasi yang Dapat Ditindaklanjuti (*Actionable Advise*):** Nilai tambah terbesar dari alat DAST ini adalah pemberian saran perbaikan (*remediation advice*) yang spesifik, sehingga *developer* dapat segera menambal celah tersebut, sesuai dengan filosofi nama "Advise".
- **Manajemen Terpusat:** Dengan adanya manajemen Target, pengguna yang mengelola banyak layanan (misalnya arsitektur *microservices*) dapat mengawasi seluruh titik akhir dari satu antarmuka terpusat.

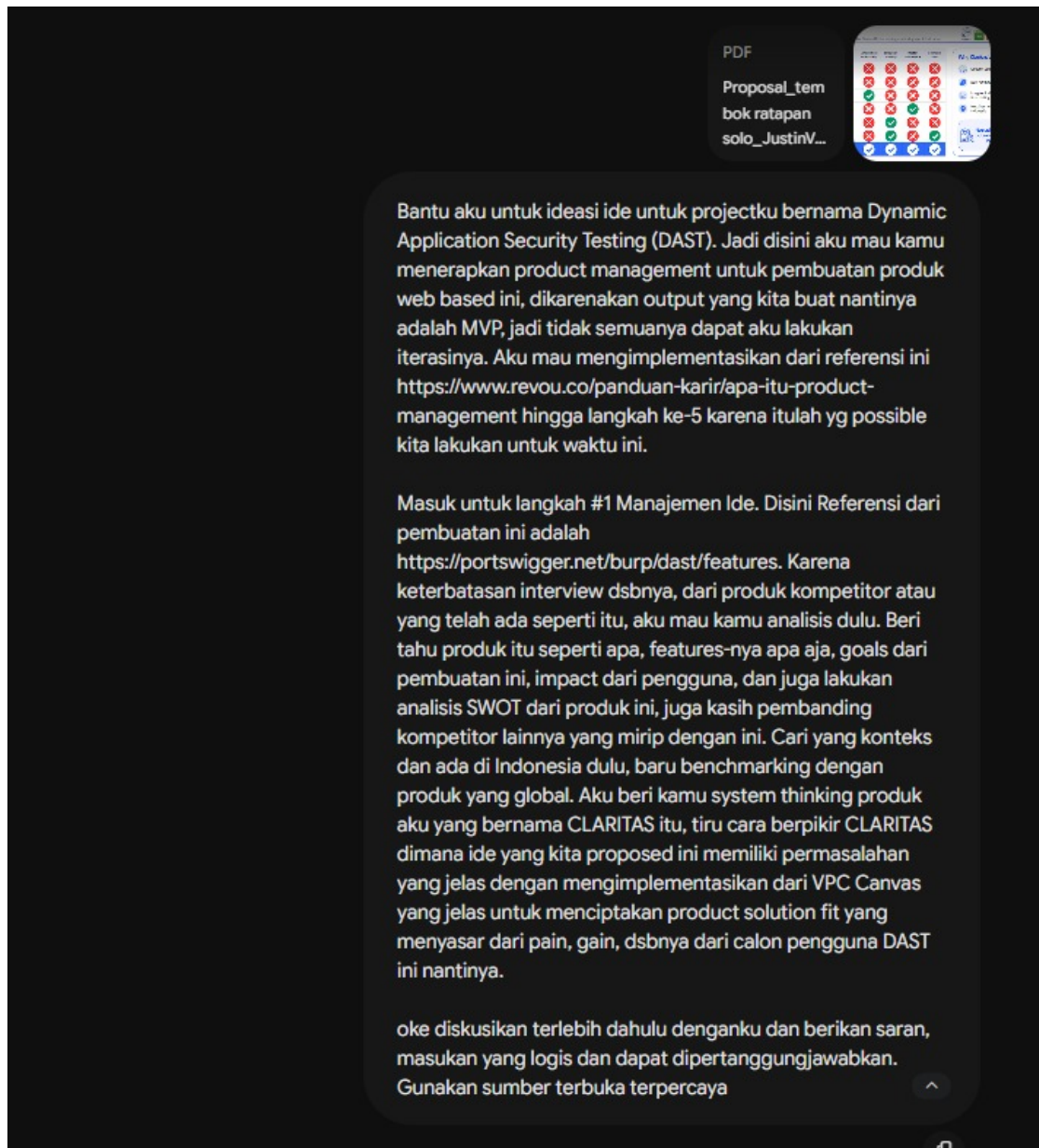
3 Metodologi AI

Dalam proses pemikiran, penyusunan arsitektur, dan penulisan kode fitur DAST "Advise", kami memanfaatkan bantuan AI untuk mempercepat siklus kerja. Berikut adalah penjelasan metodologi dan teknik *prompting* yang diterapkan secara transparan:

3.1 Ideasi dan Konseptualisasi

Pada tahap awal, kami menggunakan model AI generatif untuk merancang ide tata letak dan struktur komponen (seperti terlampir pada gambar ideasi). Kami menggunakan percakapan berbasis *chat* untuk mengklarifikasi kebutuhan pengguna dan memvisualisasikan arsitektur.

Teknik Prompting: Kami menerapkan teknik *Role-Playing* dan memberikan konteks yang spesifik (*Context-Setting*). *Prompt:* "Bertindaklah sebagai UI/UX Designer untuk aplikasi keamanan siber. Rancang struktur komponen yang logis untuk halaman Dashboard DAST, dengan fokus menampilkan tingkat keparahan (severity) kerentanan dengan sangat jelas."



Gambar 5: Proses Ideasi Menggunakan Bantuan AI

3.2 Perancangan UI/UX

Pada tahap selanjutnya, kami menggunakan *prompt* komprehensif untuk merancang arsitektur informasi dan aliran UX untuk aplikasi DAST, yang kemudian dieksekusi dalam bentuk purwarupa Figma.

Prompt yang Digunakan:

Peran: Bertindak sebagai Senior Product Designer untuk SaaS B2B.

Produk: ADVISE — DAST SaaS untuk software house, QA engineer, dan junior developer di Indonesia.

Design tokens: Primary #111844, Secondary #4B5694, Tertiary #7288AE, Surface #EAE0CF & #FFFFFF, semantic merah/oranye/kuning/biru muted untuk severity. Font Plus Jakarta Sans/Inter. Radius card 12px,

button 8px.

Instruksi: Bangun base components dulu: Button, Input Field, Severity Badge, Data Table, Card, Side Drawer, Code Block, Toast — sebelum menyusun layar.

Susun 25 frame sesuai ID & nama pada Bagian 3, dikelompokkan per modul (Onboarding, Core Scanning, Results & Insights, Project Management, Team, Account/Billing, Edge States).

Untuk setiap frame, rujuk spesifikasi Komponen UI, States, dan Catatan Build Figma pada Bagian 4 sebagai requirement layout.

Pastikan navigasi antar frame memakai Figma prototyping link sesuai kolom Navigasi Lanjutan tiap layar, agar clickable prototype bisa diuji end-to-end mengikuti 3 user journey pada Bagian 5.

Vibe akhir: profesional B2B, minim friksi kognitif, ramah untuk developer junior — hindari ilustrasi terlalu playful atau warna terlalu jenuh.

3.3 Pembuatan Kode (*Code Generation*)

Untuk implementasi *frontend*, kami menggunakan asisten AI untuk membangun struktur kode Next.js dan styling dengan Tailwind CSS. Kode implementasi dan repositori proyek selengkapnya dapat diakses pada tautan berikut: <https://github.com/shzirley/DAST-netics-project>.

Batasan (Constraints) yang Digunakan: Kami memberikan *constraints* tegas agar kode mematuhi standar produksi. *Prompt:* "Buat komponen React untuk menampilkan daftar kerentanan. Komponen harus menggunakan Tailwind CSS. Wajib menerima *props* berupa array data kerentanan (id, nama, severity). **Constraint:** Gunakan *client component* ('use client') dan pastikan desain *responsive* untuk layar kecil dan besar."